

Vite & Gourmand

Documentation technique

Projet ECF

Graduate DWWM

COLET JENNIFER

Table des matières

1. Réflexion initiales technologique	3
1.1 Choix technologiques	3
1.2 Architecture bi-base innovante	4
2. Configuration de l'environnement de travail	4
2.1 Outils utilisés	4
2.2 Structure du projet	5
3. Modèle conceptuel de données	7
Entités principales	7
Relations et cardinalités clés	8
4. Diagrammes	10
4.1 Diagramme d'utilisation	10
Acteurs	10
Cas d'utilisation principaux	10
Relation particulière	10
4.1 Diagramme de séquence	12
Objectif	12
Déroulement	12
Résultat	12
5. Déploiement	15
5.1 Préparation du projet	15
5.2 Choix de l'hébergement	15
5.3 Envoi des fichiers	16
5.4 Gestion de l'arborescence	16
5.5 Base de données MySQL	16
5.6 Modification du fichier de configuration	16
5.7 Vérification finale	17

Cette documentation technique décrit l'architecture, les choix technologiques et l'implémentation de l'application web "Vite & Gourmand, application e-commerce de commande traiteur développée en PHP/MySQL.

1. Réflexion initiales technologique

1.1 Choix technologiques

PHP 8.1

- Exécution native sur XAMPP, aucune configuration supplémentaire
- Sessions côté serveur intégrées - idéal pour le panier et l'authentification
- PDO : couche d'abstraction sécurisé pour MySQL (requête préparées)

MySQL

- Natif sur XAMPP
- Relations / clés étrangères
- Type JSON natif
- PDO inclus
- Multi-utilisateurs

JavaScript

- fetch API + polling en temps réel

Bootstrap 5.3

- Responsive + composants (navbar, modals)

CSS

- CSS organisé en 3 fichiers : global.css, public.css, espace.css

Composer

- Gestionnaire de dépendances PHP

PHPMailer

- Envoi d'email transactionnels

1.2 Architecture bi-base innovante

Tables relationnelles	users, menus, commandes, commande_items, avis, horaires
Table nosql_documents	Stockage JSON par collection (stats_menu, stats_daily)
Classe NoSQLStore	find(), findOne(), insertOne() updateOne(), upsertOne(), count()
Classe StatSync	Calcule les KPIs depuis MySQL et les persiste dans nosql_documents

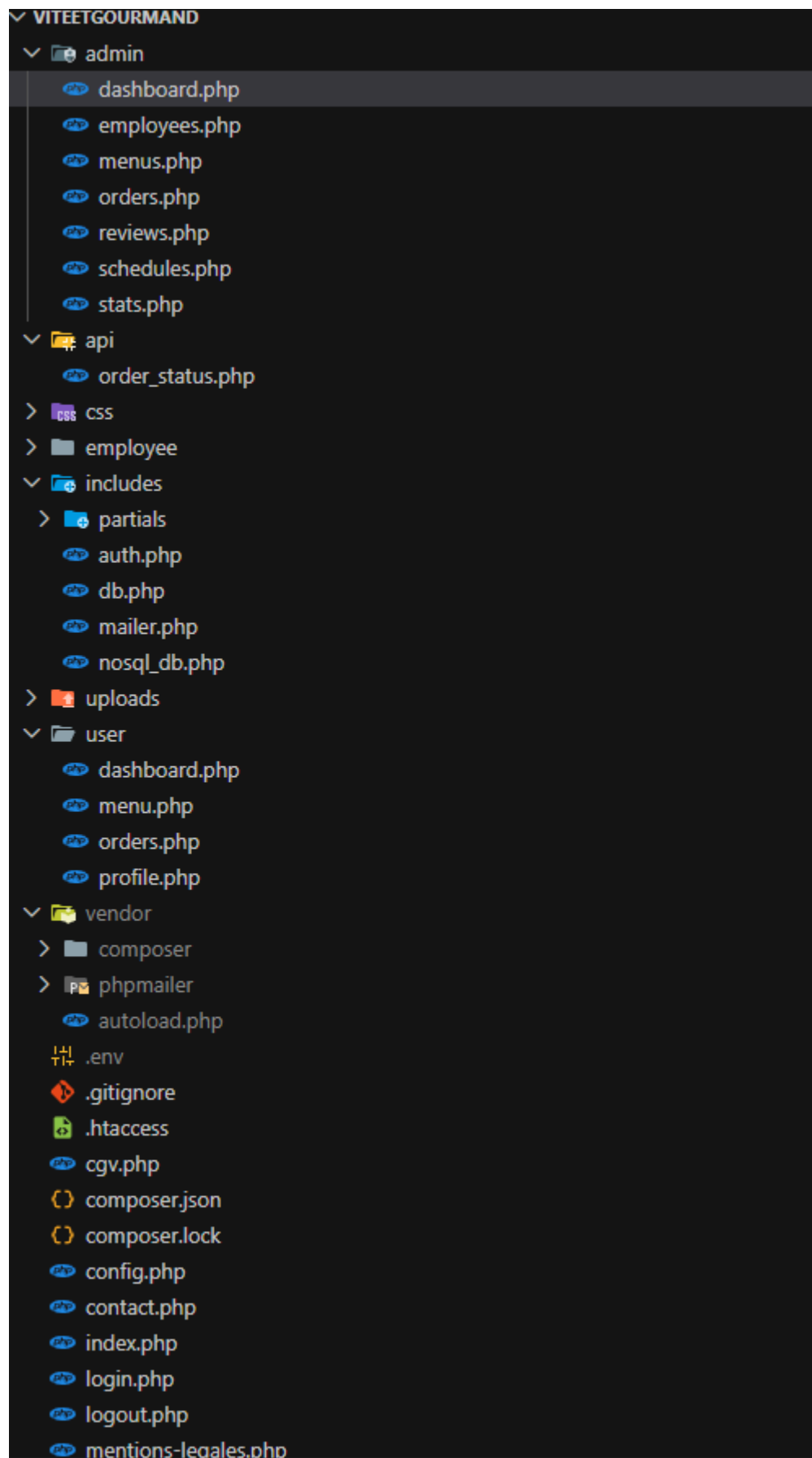
2. Configuration de l'environnement de travail

2.1 Outils utilisés

Outil	Version/détail	Rôle
XAMPP	8.1+	Serveur Apache + MySQL local - environnement principal
VS Code	Dernière	Editeur de code - extensions PHP Intelephense, Live server
phpmyadmin	Inclus Xampp	Interface graphique MySQL - import/export SQL, modifications
GIT/ GitHub		Versionnage du code source - commits par fonctionnalité
Chrome DevTools	Inclus Chrome	Débogage JS (Console, Network, Application - sessions AJAX)

2.2 Structure du projet

Fichier/Dossier	Rôle
config.php	Identifiants MySQL - seul fichier à modifier selon l'hébergeur
setup.php	Initialise la BDD, crée les tables et les données de test
vite_gourmand.sql	Export SQL complet - pour import phpMyAdmin
includes/db.php	Connexion PDO partagée par toutes les pages
includes/auth.php	Sessions, rôles, helpers (stars, statutBadge)
includes/nosql_db.php	NoSQLStore + StatsSync - architecture bi-base
api/order_status.php	Endpoint JSON pour le polling temps réel (30sec)
css/global.css	Variables CSS, navbar, bouton - toutes les pages
css/public.css	Styles pages publiques (hero, filtres, cards)
css/espace.css	Styles back-office (panier, commandes, stats)
admin/(7pages)	dashboard, menus, orders, reviews, schedules, employees, stats
employee/(5pages)	dashboard, menus, orders, reviews, schedules
user/(4pages)	dashboard, menu, orders, prfile
uploads/	Photos des menus
.htaccess	Options -Indexes - blocage scripts dans upload/



3. Modèle conceptuel de données

Le MCD de l'application Vite & Gourmand a été conçu avec le logiciel Looping. Il représente l'ensemble des entités du système et leurs relations.

Entités principales

Utilisateurs - entité centrale du système. Elle stocke les informations personnelles de chaque compte (nom, prénom, email, adresse, téléphone). Un utilisateur possède un rôle (client, employé, administrateur) via la relation **possède** avec l'entité **rôle**.

Commande - entité clé du processus métier. Elle contient toutes les informations liées à une prestation : nombre de personnes, adresse, frais de livraison, frais kilométrique, remise, prix total et le statut. Un utilisateur peut passer zéro ou plusieurs commandes (relation **passe**, cardinalité 1,1 - 0,n).

Menu - représente un menu du catalogue. Il est défini par un nom, une description, un prix, un nombre de personnes minimum et le stock disponible. Un menu :

- **appartient** à une **catégorie** (Noël, Pâques, Mariage, Classique)
- **respecte** un **régime** alimentaire (végan, végétarien..)
- **contient** un ou plusieurs **plats** via la relation **contient**

Plat - représente un plat individuel (entrée, plat, dessert) identifié par un nom et un type. Il **comporte** zéro ou plusieurs **allergènes**.

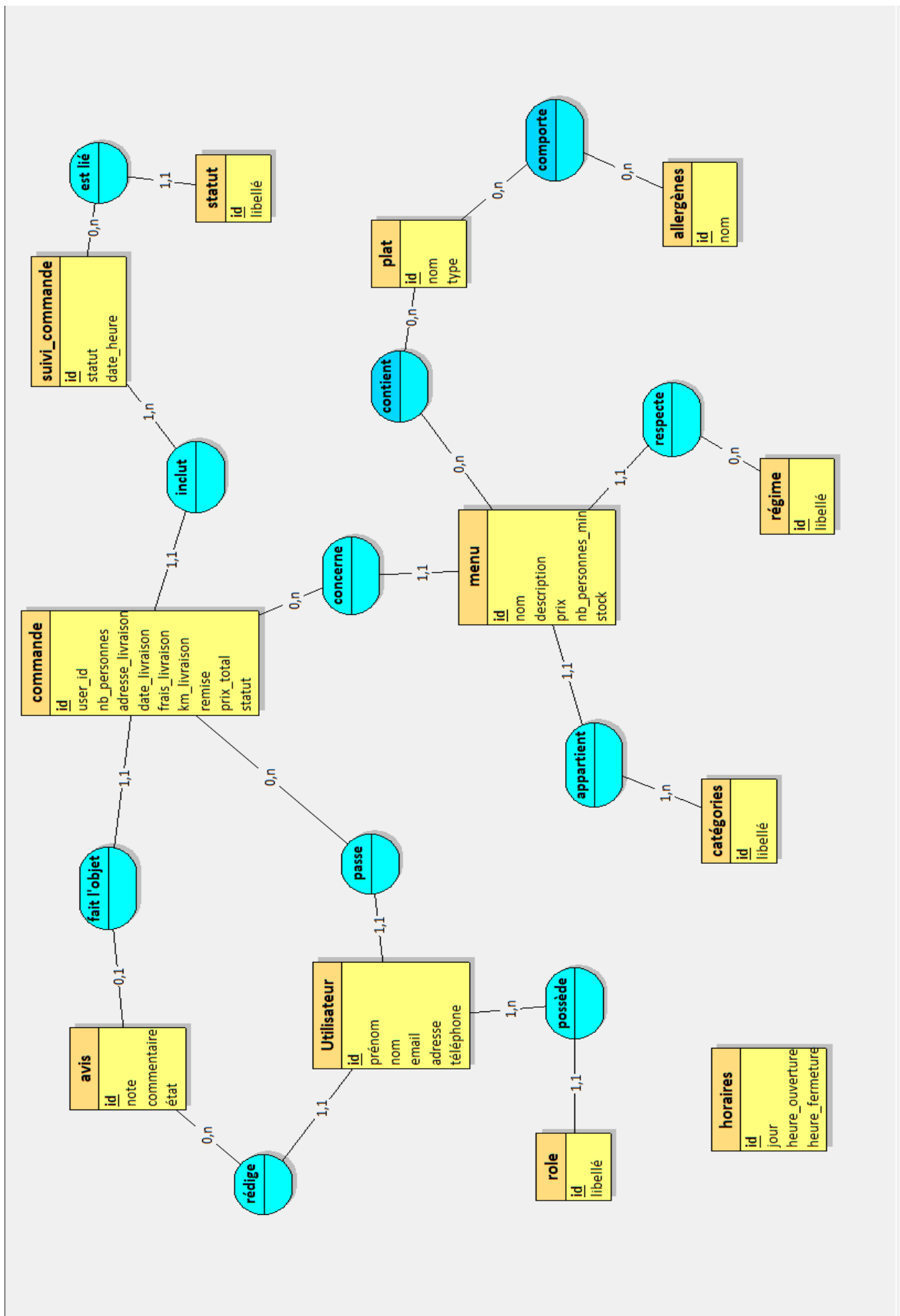
Avis - un utilisateur peut rédiger zéro ou plusieurs avis (relation **rédige**, cardinalité 1,1 - 0,n). Chaque avis contient une note, un commentaire et un état de modération. Un avis **fait l'objet** d'une commande (cardinalité 0,1 - 1,1), ce qui garantit qu'un avis est toujours lié à une prestation réelle.

Suivi_commande - entité qui permet de tracer l'historique des changements de statut d'une commande. Elle est liée à **statut** (via la relation **est lié**) et à **commande** (via la relation **inclut**, cardinalité 1,1 - 0,n). Chaque changement de statut est horodaté (date_heure).

Horaires - entité indépendante qui stocke les horaires d'ouverture de l'établissement par jour de la semaine.

Relations et cardinalités clés

Relation	Signification
Utilisateur <i>passe</i> commande (1,1 - 0,n)	Un utilisateur peut avoir plusieurs commandes, une commande appartient à un seul utilisateur
Commande <i>concerne</i> Menu (0,n - 1,1)	Une commande concerne un menu, un menu peut être dans plusieurs commandes
Commande <i>inclut</i> suivi_commande (1,1 - 0,n)	Une commande peut avoir plusieurs entrées de suivi horodatées
Menu <i>contient</i> Plat (0,n - 0,n)	Un menu peut contenir plusieurs plats, un plat peut appartenir à plusieurs menus
Plat <i>comporte</i> Allergènes (0,n - 0,n)	Un plat peut contenir plusieurs allergènes, un allergène peut être présent dans plusieurs plats
Utilisateur <i>rédige</i> Avis (1,1 - 0,n)	Un utilisateur peut rédiger plusieurs avis
Avis <i>fait l'objet</i> Commande (0,1 - 1,1)	Un avis est lié à une commande spécifique



4. Diagrammes

4.1 Diagramme d'utilisation

Acteurs

- Visiteur : consulter les menus, contacter l'entreprise, créer un compte
- Client connecté : se connecter, consulter les menus, commander, annuler une commande (sous conditions), laisser un avis
- Employé : se connecter, consulter les menus, mettre à jour les commandes, valider ou refuser les avis.
- Administrateur : gérer les comptes employés, les commandes, les plats et horaires, valider/refuser les avis et consulter les statistiques.

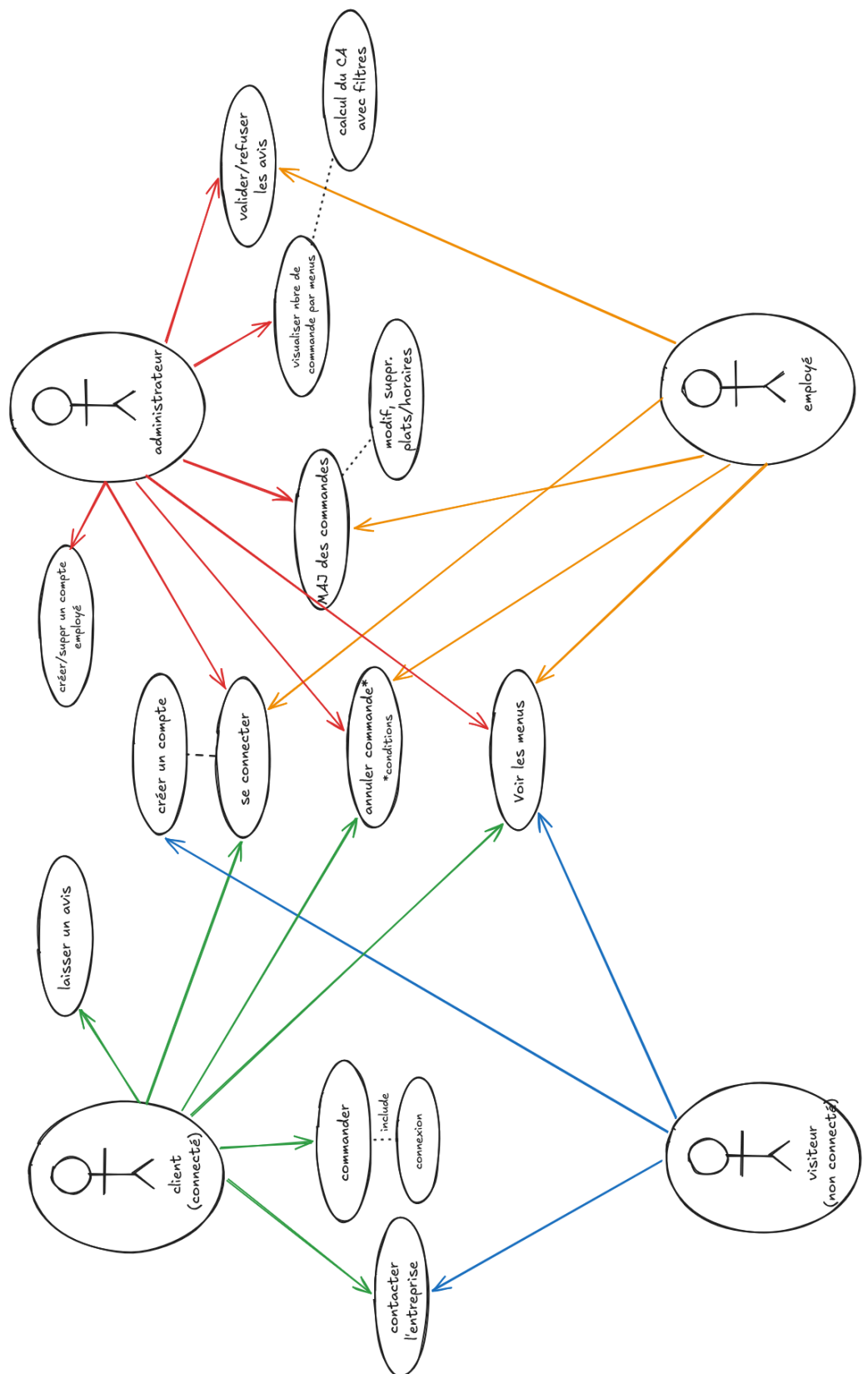
Cas d'utilisation principaux

- **Créer un compte** : permet à un visiteur de venir client
- **Se connecter** : donner accès aux fonctionnalités selon le rôle l'utilisateur
- **Voir les menus** : consultation des plats disponibles
- **Commander** : un client connecté peut passer une commande
- **Annuler une commande** : possible selon certaines conditions
- **Laisser un avis** : un client peut donner son avis après son expérience
- **Mettre à jour les commandes** : suivi de l'état des commandes par le personnel
- **Valider ou refuser les avis** : modération des avis clients
- **Gérer les plats et horaires** : administration du menu et des horaires
- **Consulter les statistiques** : visualisation du nombre de commandes et du chiffre d'affaires

Relation particulière

- La fonctionnalité **Commander** nécessite que le client soit **connecté** (<<include>>).

Vite & Gourmand



4.1 Diagramme de séquence

Séquence 1 : Connexion utilisateur

Objectif

Permettre à un utilisateur de s'authentifier afin d'accéder à son espace personnel.

Déroulement

1. L'utilisateur saisit son email et son mot de passe.
2. Le navigateur envoie les informations à *login.php*.
3. *login.php* recherche l'utilisateur dans la base de données.
4. La base de données retourne les informations de l'utilisateur.
5. Le mot de passe est vérifié avec *password_verify()*.
6. Les informations utiles (*user_id*, nom, rôle) sont enregistrées en session.
7. L'utilisateur est redirigé vers son tableau de bord.

Résultat

L'utilisateur est connecté et peut accéder aux fonctionnalités correspondant à son rôle.

Séquence 2 : Passage de commande

Objectif

Permettre à un client de passer une commande et recevoir une confirmation.

Déroulement

1. Le client ajoute des produits à son panier via AJAX.
2. Le panier est mis à jour et renvoyé au format JSON.
3. Le client valide sa commande.
4. *menu.php* enregistre la commande dans la table commandes.
5. Les articles commandés sont enregistrés dans *commande_items*.
6. L'identifiant de la commande est retourné.
7. Un email de confirmation est envoyé via PHPMailer.
8. Un message de confirmation est affiché au client.

Résultat

La commande est enregistrée en base de données et le client reçoit une confirmation.

Séquence 3 : Mise à jour statut commande (employé)

Objectif

Permettre à un employé de modifier le statut d'une commande et informer le client de la mise à jour.

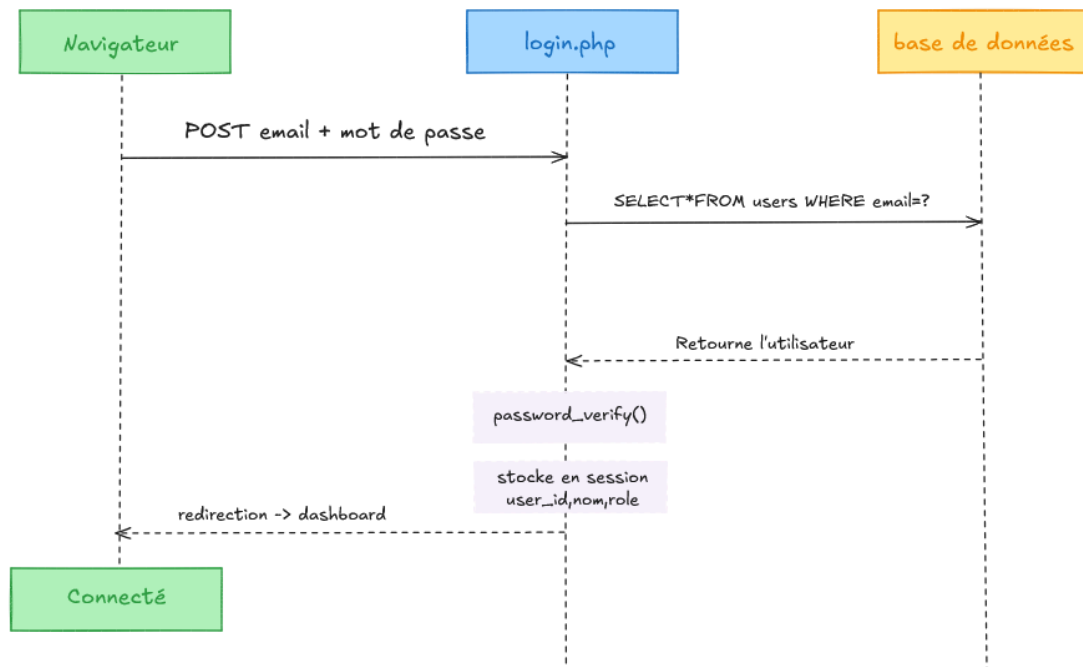
Déroulement

- L'employé sélectionne un nouveau statut pour une commande.
- *orders.php* vérifie les droits de l'utilisateur avec *checkAdminOrEmployee()*.
- Une requête UPDATE modifie le statut dans la base de données MySQL.
- La base confirme la mise à jour.
- Le client interroge régulièrement *order_status.php* via *fetch()* (toutes les 5 secondes).
- Le nouveau statut est renvoyé au format JSON.
- Le badge de statut est automatiquement mis à jour sur l'interface client.

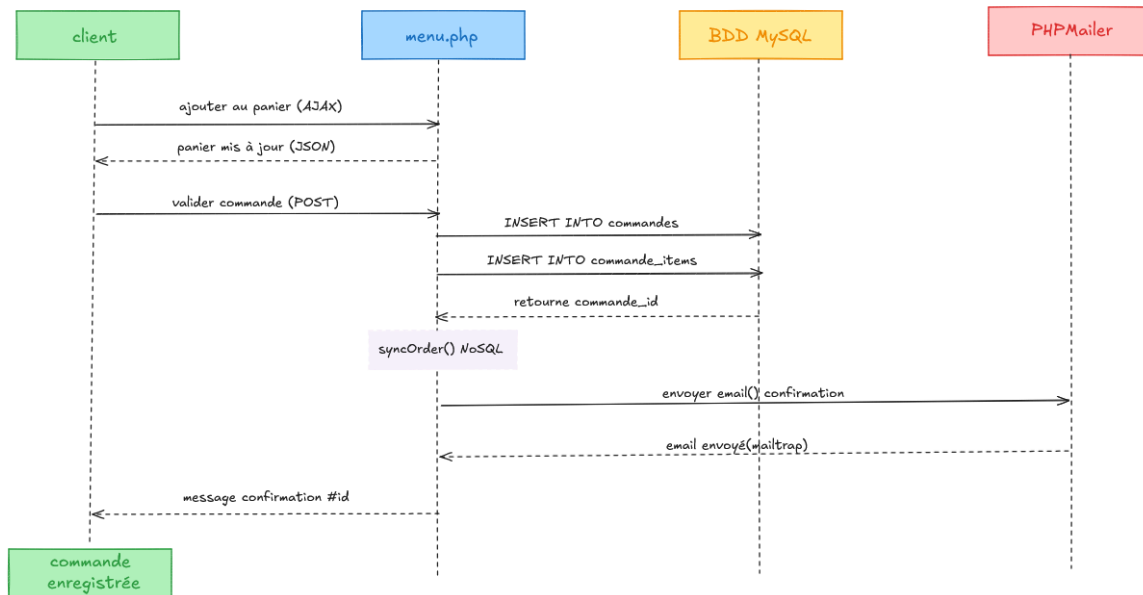
Résultat

Le statut de la commande est mis à jour en base de données et visible par le client presque en temps réel

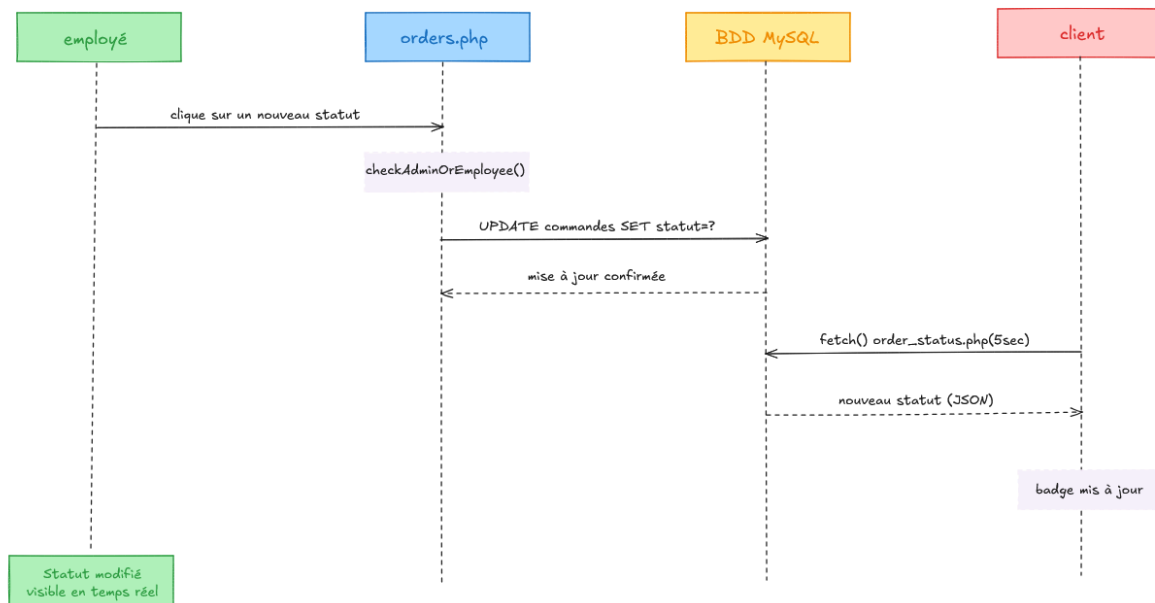
Séquence 1 - connexion utilisateur



Séquence 2 - passage de commande



Séquence 3 - mise à jour statut commande (employé)



5. Déploiement

5.1 Préparation du projet

Le site a d'abord été préparé en local avec la structure complète du projet : fichiers HTML/PHP, feuilles de style CSS, scripts JavaScript, images et fichiers de configuration nécessaires. J'ai vérifié que la page d'entrée était bien nommée `index.html` ou `index.php`, car c'est le fichier chargé en premier lors de l'ouverture du site.

5.2 Choix de l'hébergement

J'ai choisi InfinityFree comme hébergeur gratuit pour publier le site en ligne. Cette solution permet d'héberger les sites PHP/MySQL et fournit un espace de stockage ainsi qu'un gestionnaire de fichiers.

5.3 Envoi des fichiers

Une fois le compte créé, je suis allé dans le Control Panel, puis dans File Manager. Les fichiers du site ont été placés dans le dossier htdocs, qui correspond à la racine publique du site InfinityFree.

5.4 Gestion de l'arborescence

Les dossiers utiles du projet, comme css, js, images, ainsi que les fichiers nécessaires, ont été conservés dans htdocs. Le but était que le site soit accessible directement depuis le domaine principal, sans sous-dossier supplémentaire inutile.

5.5 Base de données MySQL

Pour la partie dynamique du site, une base MySQL a été créée dans la section MySQL Databases du panneau InfinityFree. Les informations de connexion utilisées dans le fichiers config.php sont :

- DB_HOST : le hostname MySQL fourni par InfinityFree
- DB_PORT : 3306
- DB_NAME : le nom de la base
- DB_USER : l'utilisateur MySQL
- DB_PASS : Le mot de passe MySQL

5.6 Modification du fichier de configuration

J'ai ensuite adapté le fichier config.php pour ajouter les valeurs fournies par InfinityFree.

```
$is_local = ($_SERVER['HTTP_HOST'] === 'localhost');

if ($is_local) {
define('DB_HOST', getenv('DB_HOST') ?: 'localhost');
define('DB_PORT', getenv('DB_PORT') ?: '3307');
define('DB_NAME', getenv('DB_NAME') ?: 'vite_gourmand');
define('DB_USER', getenv('DB_USER') ?: 'root');
define('DB_PASS', getenv('DB_PASS') ?: '');
define('BASE_URL', '/viteetgourmand/');
```



```
} else {  
    define('DB_HOST', 'sql202.infinityfree.com');  
    define('DB_PORT', '3306');  
    define('DB_NAME', 'if0_42164674');  
    define('DB_USER', 'if0_42164674_bdd');  
    define('DB_PASS', '*****');  
    define('BASE_URL', '/');  
}
```

5.7 Vérification finale

Après l'envoi des fichiers et la configuration de la base, j'ai testé le site en ouvrant l'URL fournie par InfinityFree dans le navigateur. J'ai vérifié l'affichage de la page d'accueil, le chargement des feuilles de style, des scripts et la connexion à la base de données.

<https://viteetgourmand.fredev.app/>